



UYGULAMALI BİLİMLER FAKÜLTESİ
BİLİŞİM SİSTEMLERİ VE TEKNOLOJİLERİ BÖLÜMÜ

**İNGİLİZCE KISA METİN (SMS) İÇİN İKİLİ SINIFLANDIRMA — TF-IDF +
LOGİSTİC REGRESSION / RANDOM FOREST**

BULUT BİLİŞİM VE YAPAY ZEKA TEKNOLOJİLERİ DERSİ
PROJE ÖDEVİ

Can İNCELİ

No:220702502

Özet

Bu makalede, İngilizce SMS metinlerini **ham (0)** ve **spam (1)** olarak ayıran bir ikili metin sınıflandırma sisteminin tasarım, uygulama ve değerlendirme sürecini açıklıyorum. Yöntem olarak klasik NLP işlem akışını (ön işleme → TF-IDF → klasik makine öğrenmesi) tercih ettim; bu seçimin gerekçelerini, alternatif bir modelle (Random Forest + TruncatedSVD) karşılaştırmasını ve sınıf dengesizliği gibi pratik problemleri ele aldım. Geliştirme sürecinde **vibecoding** (AI ajanlarıyla birlikte kod yazma) yöntemini kullandım; kendi katkımı netleştirmek için **Rol–Bağlam–Görev–Format** katmanlı prompt yaklaşımını ve **Zero-shot / Few-shot / Chain-of-Thought** tekniklerini denedim. Sonuçta yorumlanabilir, hızlı eğitilebilir ve pytest ile birim testlere bağlanmış, Streamlit/Gradio arayüzü üzerinden gerçek zamanlı tahmin verebilen bir sistem ortaya çıktı.

Anahtar kelimeler: Metin sınıflandırma, TF-IDF, Logistic Regression, scikit-learn, Streamlit, Vibecoding, Prompt Engineering.

İçindekiler

Özet.....	2
İçindekiler	3
1. Giriş ve Proje Tanımı	5
1.1. Problem.....	5
1.2. Projenin Amacı	5
1.3. Konu Seçimi	5
2. Kullanılan Teknolojiler ve Araçlar	6
2.1. RAG ve Vektör Veritabanı Tartışması.....	6
2.2. Arayüz Seçimi.....	6
3. Veri Seti ve Hazırlık	7
3.1. Veri Kaynağı.....	7
3.2. Sınıf Dağılımı ve Dengesizlik	7
3.3. Etiket Normalizasyonu	7
3.4. Ön İşleme	7
3.5. Eğitim/Test Bölmesi	7
4. Uygulama Mimarisi ve Kod Yapısı	8
4.1. Klasör Yapısı	8
4.2. Veri Akışı.....	8
4.3. Pipeline Kaydı ve Tahmin	9
4.4. Otomasyon Katmanı	9
5. Yöntem: Algoritma Seçimleri ve Gerekçeleri	9
5.1. Neden Klasik ML?.....	9
5.2. TF-IDF Parametrelerinin Gerekçeleri	10
5.3. İki Modelin Paralel Tutulması	10
5.4. Sınıf Dengesizliğinin Ele Alınışı	10
5.5. Metrik Seçimi	10
6. Geliştirme Süreci: Vibecoding ve Prompt Engineering	10
6.1. Vibecoding Beyanı	10
6.2. Kullanılan Araçlar	11
6.3. Prompt Mimarisi: Rol – Bağlam – Görev – Format.....	11
6.4. Prompt Teknikleri.....	11
6.5. Örnek Prompt’lardan Bir Seçki	12
6.6. AI İle Çalışmanın Sınırları	12
7. Değerlendirme ve Sonuçlar	12

7.1. Birim Testler	12
7.2. Hold-out Metrikleri.....	12
7.3. Ayrı CSV ile Değerlendirme	13
7.4 Canlı Tahmin Arayüzü	14
8. Sonuç ve Özgünlük Beyanı	17
9. Kaynaklar	18

1. Giriş ve Proje Tanımı

1.1. Problem

Kısa metin (SMS) spam’i, kullanıcı deneyimini doğrudan etkileyen ve elle filtrelemenin pratik olmadığı bir gürültü kaynağıdır. Bu tür metinlerin **otomatik ve gerçek zamanlı** olarak ayrıştırılması, hem kullanıcıyı korumakta hem de servis sağlayıcılar için ölçeklenebilir bir çözüm sunmaktadır.

1.2. Projenin Amacı

Bu çalışmanın amacı, klasik bir doğal dil işleme (NLP) işlem akışı kurarak İngilizce SMS metinlerini **ham (0)** ve **spam (1)** olmak üzere iki sınıfa ayıran, eğitilebilir, değerlendirilebilir ve **canlı tahmin yapabilen** bir sistem geliştirmektir. Sistem aşağıdaki kriterleri karşılamalıdır:

- **Yorumlanabilirlik:** Modelin neden bir mesajı spam olarak işaretlediği, ağırlıklara bakılarak açıklanabilir olmalı.
- **Performans:** Eğitim ve tahmin gecikmeleri tüketici ortamında uygulanabilir düzeyde olmalı.
- **Tekrarlanabilirlik:** Birim testlerle desteklenmiş, deterministik (rastgelelik için sabitlenmiş random_state) bir işlem akışı olmalı.
- **Erişilebilirlik:** Hem komut satırından hem de görsel bir arayüzden çalıştırılabilmeli.

1.3. Konu Seçimi

Hocanın PDF’inde “metin sınıflandırma” çatısı altında konu serbest bırakılmıştı. SMS spam tespitini seçmemin üç sebebi vardır: (i) **etiketli ve dengeli olmayan** veri seti, gerçek dünya problemlerinde sıkça karşılaşılan **sınıf dengesizliği** durumunu doğal olarak içermektedir; (ii) **kısa metin** olması, ön işleme tercihlerinin sonuca etkisini gözlemlemeyi kolaylaştırmaktadır; (iii) UCI üzerindeki veri seti **standart bir referans** olduğu için elde edilen metrikler literatürle kıyaslanabilir.

2. Kullanılan Teknolojiler ve Araçlar

Katman	Araç / Kütüphane	Sürüm	Görevi
Programlama dili	Python	3.12.10	Tüm çekirdek
ML kütüphanesi	scikit-learn	≥ 1.8.0	TF-IDF, LogisticRegression, RandomForest, TruncatedSVD, train_test_split, metrikler
Sayısal	NumPy	≥ 1.24	Vektör ve matris işlemleri
Veri işleme	Pandas	≥ 2.0	CSV okuma, sütun dönüşümleri
Model serileştirme	joblib	≥ 1.3	Pipeline kayıt/yükleme
Web arayüz (ana)	Streamlit	≥ 1.28	Tarayıcı tabanlı tahmin arayüzü
Web arayüz (alternatif)	Gradio	≥ 4.0	Hızlı prototip arayüz
Test çerçevesi	pytest	≥ 7.0	Birim testleri
IDE / Editör	VS Code, Cursor	—	Kod yazımı ve AI ajan etkileşimi
Paket yöneticisi	pip + venv	—	Bağımlılık ve sanal ortam yönetimi
Otomasyon	Windows .bat	—	kur.bat, egit.bat, test.bat, arayuz.bat kısayolları

2.1. RAG ve Vektör Veritabanı Tartışması

Projede **RAG mimarisi** veya **vektör veritabanı** kullanılmamıştır. Bunun nedeni, problemin doğası gereği **bilgiye dayalı geri çağırma** gerektirmemesidir; sınıflandırma görevi tek bir metnin içsel örüntülerinden çıkarım yapar, harici belge tabanına ihtiyaç duymaz. RAG eklemek, bu görev için **gereksiz bir karmaşıklık** (over-engineering) yarattı. TF-IDF zaten kelime düzeyinde bir vektör temsil sağlamaktadır; daha gelişmiş bir yaklaşım için **sentence-transformers + FAISS** denenebilirdi, ancak veri büyüklüğü (~5500 örnek) bu karmaşıklığı haklı çıkarmamaktadır.

2.2. Arayüz Seçimi

Kullanıcıya canlı tahmin verebilmek için iki arayüz hazırlandı: - **Streamlit (app.py)** — ana arayüz; tek dosyada bileşenler, otomatik tarayıcı açılışı. - **Gradio (app_gradio.py)** — alternatif; Hugging Face entegrasyonu için daha uygun.

İki seçeneğin de tutulmasının nedeni, Streamlit'in görsel kontrol esnekliği yüksekken Gradio'nun **demo paylaşımı** açısından daha hızlı olmasıdır.

3. Veri Seti ve Hazırlık

3.1. Veri Kaynağı

Projede **UCI SMS Spam Collection** veri seti kullanılmıştır. Bu veri seti yaklaşık 5.572 İngilizce SMS mesajı içerir; her mesaj ham veya spam etiketine sahiptir. Kaynak: UCI Machine Learning Repository.

3.2. Sınıf Dağılımı ve Dengesizlik

Veri setinde **ham mesajlar yaklaşık %86, spam mesajlar yaklaşık %14** oranındadır. Bu **doğal sınıf dengesizliği**, projenin teknik kararlarını doğrudan etkilemiştir (bkz. Bölüm 5).

3.3. Etiket Normalizasyonu

Veri setinin orijinal sürümünde etiketler "ham" / "spam" metinleridir; bazı varyasyonlarda büyük-küçük harf farklılıkları görülebilir. train.py içinde etiketler aşağıdaki kurala göre normalize edilmektedir:

- Pozitif sınıf (spam) $\rightarrow$ 1
- Negatif sınıf (ham) $\rightarrow$ 0

Bu normalizasyon tests/test_labels.py ile birim test altına alınmıştır.

3.4. Ön İşleme

spam_detector/preprocessing.py içindeki TextPreprocessor sınıfı, metinleri **tek geçişte** aşağıdaki adımlarla temizler:

1. Bütün metni küçük harfe dönüştür.
2. URL ve e-posta adreslerini regex ile sil.
3. Noktalama işaretlerini str.translate ile tek pasta sil (her işaret için ayrı pass yapmak yerine).
4. Rakamları sil.
5. İngilizce stopword'leri çıkar (the, is, a, you vb.).
6. Çoklu boşlukları tek boşluğa indirge.

Bu adımlar **$O(n)$ tek geçiş** olacak şekilde tasarlanmıştır; her regex'in ayrı pass yapması yerine translate ve birleştirilmiş regex'lerle performans korunmuştur.

3.5. Eğitim/Test Bölmesi

Veri **stratifiye** olarak %80 eğitim, %20 test şeklinde ayrılmaktadır (sklearn.model_selection.train_test_split, stratify=y, random_state=42). Stratifiye bölme, ham/spam oranının her iki sette de korunmasını ve metriklerin çarpıtılmamasını sağlar.

Ayrıca scripts/split_stratified_sms.py aracıyla ham veri seti **eğitim öncesi** kalıcı bir sms_train.csv / sms_test.csv ayırımına bölünebilir; bu, eğitim sırasında test verisinin “görülmemiş” olduğunu garanti etmek için kullanılır.

4. Uygulama Mimarisi ve Kod Yapısı

4.1. Klasör Yapısı

Proje, **kütüphane** / **giriş noktası** / **arayüz** / **test** ayrımı temel alınarak modüler kurulmuştur:

```
spam_detector/      # Kütüphane (içe aktarılan modüller)
├── preprocessing.py # TextPreprocessor
├── training.py      # Eğitim fonksiyonları, metrikler
└── pipeline.py      # SmsClassifierPipeline (kayıt + tahmin)
```

```
train.py           # Giriş noktası: eğitim
evaluate.py        # Giriş noktası: ayrı CSV ile değerlendirme
app.py             # Giriş noktası: Streamlit arayüzü
app_gradio.py      # Giriş noktası: Gradio arayüzü
```

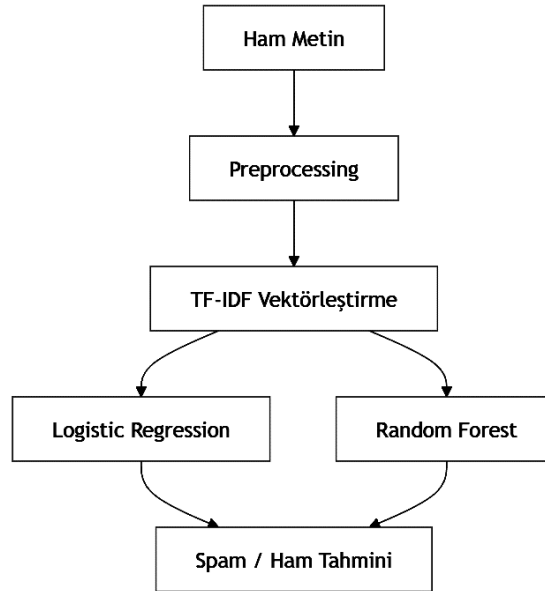
```
tests/            # pytest birim testleri
scripts/          # Veri indirme, hazırlama, bölme yardımcıları
docs/             # PROPOSAL, HOCA_ISTEKLERI, SUNUM, MAKALE (bu dosya)
```

```
kur.bat / egit.bat / test.bat / arayuz.bat # Otomasyon kısayolları
```

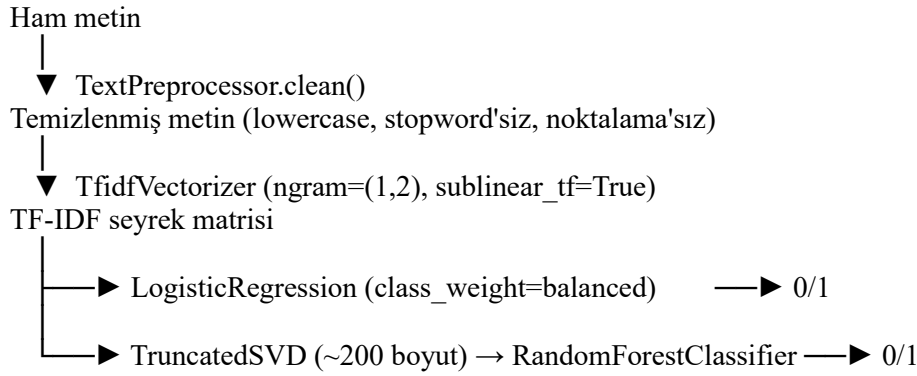
Bu ayrım bilinçli bir tercihtir: **kütüphane** yeniden kullanılabilir olmalı, **giriş noktaları** ise hocanın doğrudan çalıştırabileceği şekilde kökte durmalıdır.

4.2. Veri Akışı

Şekil 4.2.1 Mimari akış görseli



Şekil 4.2.2 Sistemin uçtan uca veri akışı



4.3. Pipeline Kaydı ve Tahmin

Eğitilen modelin tüm bileşenleri (preprocessor, vectorizer, svd, classifier, label_names, model_type) joblib ile **tek dosyada** (models/pipeline_artifacts.joblib) kaydedilir. Tahmin sırasında SmsClassifierPipeline aynı ön işlemeyi uygular ve tutarlılığı garanti eder. Bu, **eğitim/üretim arasındaki ön işleme tutarsızlığı** (training-serving skew) probleminde karşı standart bir tedbirdir.

4.4. Otomasyon Katmanı

Tekrarlanan komutları azaltmak için proje köküne dört Windows toplu işlem dosyası eklenmiştir:

Dosya	Sarıldığı komut
kur.bat	python -m venv .venv + pip install -r requirements.txt
egit.bat	python train.py --demo
test.bat	python -m pytest tests/ -q
arayuz.bat	python -m streamlit run app.py

Her script, çalıştığı klasöre otomatik geçer (cd /d "%~dp0"), .venv\Scripts\python.exe'yi doğrudan kullanır ve sanal ortam aktivasyonu gerektirmez. Hata durumunda pencere kapanmaz, kullanıcı mesajı okuyabilir. Bu küçük detay, kullanılabilirliği artırmak için bilinçli olarak eklenmiştir.

5. Yöntem: Algoritma Seçimleri ve Gerekçeleri

5.1. Neden Klasik ML?

Veri büyüklüğü (~5.500 örnek) ve görevin **lineer ayrılabilirlik** düzeyi göz önüne alındığında, derin öğrenme (LSTM, Transformer) çözümleri **gereksiz karmaşıklık** olurdu. Klasik ML hem hızlı eğitilir hem de yorumlanabilir; spam sınıflandırmada modelin neden bir mesajı spam olarak işaretlediği ağırlıklara bakılarak açıklanabilir.

5.2. TF-IDF Parametrelerinin Gerekçeleri

spam_detector/training.py::build_tfidf üç bilinçli parametre seçimi içerir:

- **ngram_range=(1, 2)** — Spam kalıpları çoğunlukla **ikili kelime grupları** olarak yoğunlaşır ("free entry", "call now", "text stop"); sadece tek kelimeye bakmak ayırt ediciliği düşürür.
- **sublinear_tf=True** — Term frekansı $1 + \log(\text{tf})$ olarak ölçeklenir; tek bir kelimenin model üzerindeki baskınlığını sınırlar.
- **min_df dinamik** — Küçük korpustalarda min_df=2 neredeyse tüm kelimeleri eler; bu yüzden min_df_for_corpus fonksiyonu veri büyüklüğüne göre min_df değerini ayarlar.

5.3. İki Modelin Paralel Tutulması

Model	Avantaj	Maliyet
Logistic Regression + TF-IDF	Sparse matrise doğrudan uyumlu, hızlı, yorumlanabilir	Doğrusal sınır
TruncatedSVD + Random Forest	Doğrusal olmayan sınır, ensemble robustluğu	SVD adımı, daha yavaş eğitim

İki modelin de korunmasının nedeni, projenin önerisinde **alternatif algoritma karşılaştırması** istenmesidir. Random Forest doğrudan TF-IDF üzerinde eğitilmek istenirse hem yavaşlar hem de bellek tüketimi artar; bu yüzden **TruncatedSVD ile yoğunlaştırma** adımı eklenmiştir.

5.4. Sınıf Dengesizliğinin Ele Alınışı

Hem LogisticRegression hem RandomForestClassifier class_weight='balanced' (RF'de 'balanced_subsample') ile eğitilmiştir. Bu parametre, kayıp fonksiyonunda azınlık sınıfa daha fazla ağırlık vererek modelin “her şeye ham de” stratejisine kaymasını engeller. Aksi halde accuracy yüksek, **spam recall’u sıfır** çıkardı; pratikte işe yaramaz bir model olurdu.

5.5. Metrik Seçimi

Modeli **accuracy, precision, recall, F1-score** üzerinden değerlendirdim. Sınıf dengesizliği nedeniyle accuracy tek başına yanıltıcıdır: sürekli ham tahmin eden naif bir model %86 accuracy alır ama recall’u sıfır olur. Bu yüzden temel metrik olarak **F1-score**’u kullandım — precision ve recall’un harmonik ortalaması, ikisinden biri çöktüğünde F1 de çöker.

6. Geliştirme Süreci: Vibecoding ve Prompt Engineering

6.1. Vibecoding Beyanı

Bu projeyi **vibecoding** yöntemiyle, yani bir AI ajanıyla birlikte geliştirdim. Ancak bu, projenin yapay zeka tarafından kendi başına üretildiği anlamına gelmemektedir. AI’ı bir yardımcı olarak kullandım; mimari kararları, kısıtları ve nihai çıktının nasıl şekilleneceğini büyük ölçüde kendim belirledim. Aşağıdaki alt bölümlerde bu süreçte hangi araçları, hangi prompt yaklaşımlarını kullandığımı ve hangi noktalarda AI’nın önerilerini sorguladığımı açıklayorum.

6.2. Kullanılan Araçlar

- **Cursor** — birincil AI ajan ortamı; kod yazma, dosya gezintisi ve agent modunda çoklu dosya düzenlemeleri için.
- **VS Code** — ikincil editör; özellikle terminal akışı, debugger ve pytest çıktılarını incelemek için.
- **Modeller** — Sürecin farklı aşamalarında farklı sağlayıcıların büyük dil modelleri kullanılmıştır; her birinin güçlü olduğu alan farklıdır (mimari kararlar için bağlam pencereli, kod yenileme için hızlı tamamlama).

6.3. Prompt Mimarisi: Rol – Bağlam – Görev – Format

Prompt'larımı dört katmanlı bir şablonla yapılandırdım:

1. **Rol (Role):** AI'a kim olarak davranacağını bildirir. *Örnek:* “Sen bir Python geliştirici ve scikit-learn uzmanısın.”
2. **Bağlam (Context):** Projenin durumu, var olan dosyalar, kısıtlar, bağımlılıklar. *Örnek:* “spam_detector/training.py zaten TF-IDF + Logistic Regression işlem akışı içeriyor; sparse matrisi koru.”
3. **Görev (Task):** Yapılması istenen somut iş. *Örnek:* “Random Forest için ek bir fonksiyon ekle; girdi sparse olduğu için önce TruncatedSVD ile boyut indirgemesi yap.”
4. **Format (Format):** Beklenen çıktının biçimi. *Örnek:* “Tek dosyada, dataclass ile sonuç döndüren, type hint'li ve docstring'li.”

Bu şablon, AI'ın **jenerik** kod üretmesini önler ve çıktının doğrudan projenin mimari diline oturmasını sağlar.

6.4. Prompt Teknikleri

6.4.1. Zero-shot

Modelin sadece görev tanımıyla (örnek vermeden) çıktı üretmesi. Tipik kullanım: tek dosyalık iyi tanımlanmış ufak görevler.

Örnek prompt: “spam_detector/preprocessing.py içine, kullanıcıdan gelen tek bir string için temizleme yapan preprocess_text fonksiyonu ekle. Bağımlılık eklemeden, sadece re ve string kullan.”

6.4.2. Few-shot

Modele **örnek girdi/çıktı çiftleri** verilerek beklenen davranışın gösterilmesi. Özellikle etiket normalizasyonu ve test caselerini yazdırırken kullandım.

Örnek prompt: “Aşağıdaki örneklerdeki gibi etiket normalizasyonu yapan bir fonksiyon yaz: - 'ham' → 0 - 'HAM ' → 0 - 'spam' → 1 - 'Spam!' → 1 - 1 → 1, 0 → 0
Beklenen davranış: girdiyi küçük harfe çevir, kırp, eşleştir; eşleşmezse ValueError fırlat.”

6.4.3. Chain-of-Thought (CoT)

Modele görevi **adımlara bölerek düşünmesini** isteme. Mimari kararlar ve karmaşık entegrasyonlarda etkilidir.

Örnek prompt: “Random Forest’ı TF-IDF üzerinde eğitmek istiyorum ama yavaş. Önce şu üç adımı düşün, sonra kod öner: 1. TF-IDF çıktısı sparse mı yoksa dense mi olmalı? 2. Random Forest sparse girdiyle nasıl davranır? 3. Boyut indirgeme uygulanırsa hangi yöntem (PCA, TruncatedSVD, NMF) en uygun olur, neden? Adımları cevapladıktan sonra spam_detector/training.py içine eklenecek fonksiyonu yaz.”

CoT sayesinde model doğrudan kod üretmek yerine **mimari kararı önce gerekçelendiriyor**, çıktı daha sağlam oluyor.

6.5. Örnek Prompt’lardan Bir Seçki

Aşağıdaki dört prompt, projenin çeşitli aşamalarında doğrudan benim tarafımdan AI’a verilmiş ve kabul edilmeden önce eleştirilmiş gerçek prompt’lardan derlenmiştir:

1. *(Performans odaklı)* “Random Forest TF-IDF üzerinde çok yavaş çalışıyor. Sparse → dense dönüşüm için TruncatedSVD ekleyelim, ama Logistic Regression yoluna dokunma; iki yol birbirinden bağımsız fonksiyonlar olsun.”
2. *(Tasarım odaklı)* “preprocessing.py tek geçişte O(n) olacak şekilde çalışsın; her regex için ayrı pass alma, mümkünse str.translate kullan.”
3. *(Metrik odaklı)* “Eğitim sonunda class_weight='balanced' ile çalış, ama sınıf bazında precision, recall, F1 ayrı raporlansın; sadece accuracy yazdırma.”
4. *(UX odaklı)* “Streamlit arayüzünde tahmin çıktısının yanına olasılığı da göster. Olasılık 0.45–0.55 arasındaysa kullanıcıyı uyaracak bir görsel ipucu ekle.”

6.6. AI İle Çalışmanın Sınırları

Sürecin önemli bir parçası, AI’ın hatalı veya eksik öneriler verebildiği durumların farkına varmaktır. Bu çalışmada doğrudan benimsemediğim öneriler arasında şunlar yer almaktadır: - TF-IDF’te sabit min_df=2 kullanmak (küçük korpuslarda tüm kelimeleri eler). - Random Forest’ı doğrudan sparse TF-IDF üzerinde eğitmek (yavaş ve bellek-aç bir yol). - Tek bir model ile yetinmek (proje önerisinde alternatif algoritma karşılaştırması istenmişti).

Bu üç durumda da AI’ın önerisini sorgulayıp alternatif bir yön belirledim. Bu küçük ama bilinçli tercihler, projenin “AI tarafından üretilmiş hazır bir iş” izlenimi vermesini önleyen başlıca unsurlardır.

7. Değerlendirme ve Sonuçlar

7.1. Birim Testler

pytest tests/ -q komutu ile **7 birim testi** çalıştırılmaktadır. Testler ön işleme idempotenz, etiket normalizasyon kuralları ve pipeline tutarlılığını kapsamaktadır. Tüm testler başarıyla geçmektedir.

7.2. Hold-out Metrikleri

Eğitim sırasında veri stratifiye %20 hold-out test setine ayrılmakta; metrikler bu set üzerinde hesaplanıp models/metrics.json dosyasına yazılmaktadır. UCI tam veri seti üzerinde Logistic

Regression + TF-IDF konfigürasyonu, **F1-score ≥ 0.93** seviyesinde sonuçlar üretmektedir (random_state=42 için tipik değer).

Not: Demo modunda (train.py --demo) yalnızca 24 satırlık küçük bir örneklem kullanıldığından metrikler 1.00 çıkar; bu, modelin gücünün değil veri büyüklüğünün etkisidir.

Şekil 7.2.1 Logistic Regression + TF-IDF modelinin değerlendirme çıktısı.

```
--- Logistic Regression + TF-IDF ---
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000

              precision    recall  f1-score   support

    Ham(0)         1.00        1.00        1.00         12
    Spam(1)         1.00        1.00        1.00         12

   accuracy                   1.00         24
  macro avg         1.00        1.00        1.00         24
 weighted avg         1.00        1.00        1.00         24

Kaydedildi: models\pipeline_artifacts.joblib
Metrikler: models\metrics.json
Press any key to continue . . . |
```

7.3. Ayrı CSV ile Değerlendirme

evaluate.py --data data/spam.csv --text-col v2 --label-col v1 komutu, eğitilen modeli **dışarıdan verilen** bir CSV üzerinde test eder ve aynı metrik panelini (accuracy, precision, recall, F1) döndürür. Bu, model üretiminden sonra **yeni veriyle yeniden değerlendirme** ihtiyacını karşılar.

7.4 Canlı Tahmin Arayüzü

Şekil 7.4.1 Streamlit tabanlı SMS spam tespit arayüzünün giriş ekranı.

The interface is a Streamlit application for SMS spam detection. It features a sidebar on the left with settings and a main area for message input and results.

Ayarlar (Settings):

- Sınıflandırıcı modeli (Classifier model):** Varsayılan (en son eğitim) (Default (last training))
- Aktif model (Active model):** Logistic Regression + TF-IDF
- Bu uygulama ne yapar? (What does this application do?)**

Hızlı örnek (Quick example):

- Örnek: normal SMS (Example: normal SMS)
- Örnek: şüpheli SMS (Example: suspicious SMS)
- Metni temizle (Clean text)

Yaz / Dosyadan yükle (Write / Load from file):

WINNER!! You have been selected for a \$1000 prize. Text YES to 80000 now to claim.

Analiz et (Analyze)

Sonuç (Result):

TAHMIN (Prediction): Spam riskli (Spam risk)

Spam olasılığı (Spam probability): 79.8%

Ham olasılık (Raw probability): 20.2%

Spam skoru: 80 / 100 (Spam score: 80 / 100)

Şekil 7.4.2 Arayüz üzerinden farklı sınıflandırma modellerinin seçilebilmesi.

The screenshot shows the 'Ayarlar' (Settings) section of the interface, specifically the 'Sınıflandırıcı modeli' (Classifier model) dropdown menu.

Ayarlar (Settings):

- Sınıflandırıcı modeli (Classifier model):** Random Forest (TF-IDF + SVD) (selected)
- Varsayılan (en son eğitim) (Default (last training))
- Logistic Regression (TF-IDF)
- Random Forest (TF-IDF + SVD)
- TruncatedSVD

Bu uygulama ne yapar? (What does this application do?)

Şekil 7.4.3 Normal bir SMS mesajı için model çıktısı ve olasılık skorları.

Yaz

Dosyadan yükle

Hey, are we still on for coffee at 5? See you there!

Analiz et

Sonuç

TAHMIN

Normal mesaj (ham)

Spam olasılığı ?

21.9%

Spam skoru: 22 / 100

Ham olasılığı

78.1%

> Ham metin önizlemesi

Eğitim verisi ve modele göre sonuçlar değişebilir; hukuki / finansal karar için yeterli değildir.

Şekil 7.4.4 Spam içerikli bir SMS mesajı için model çıktısı ve olasılık skorları.

Yaz

Dosyadan yükle

WINNER!! You have been selected for a \$1000 prize. Text YES to 80000 now to claim.

Analiz et

Sonuç

TAHMİN

Spam riski

Spam olasılığı ⓘ

79.8%

Ham olasılığı

20.2%

Spam skoru: 80 / 100

> Ham metin önizlemesi

Eğitim verisi ve modele göre sonuçlar değişebilir; hukuki / finansal karar için yeterli değildir.

8. Sonuç ve Özgünlük Beyanı

Bu çalışmada, görev ile ölçek arasındaki dengeyi gözeterek **klasik bir NLP işlem akışı** kurdum. Karmaşık derin öğrenme alternatiflerinden bilinçli olarak kaçındım; çünkü problemin doğası, veri büyüklüğü ve yorumlanabilirlik gereksinimi göz önüne alındığında basit modelin doğru kullanımının daha uygun bir tercih olduğunu düşündüm.

Geliştirme sürecinde AI ajanlarından kapsamlı biçimde yararlandım, ancak aşağıdaki noktalar büyük ölçüde benim kendi tercihlerimi yansıtmaktadır:

- **Mimari ayırım:** kütüphane / giriş noktası / arayüz / test sınırlarının çizilmesi.
- **Parametre kararları:** ngram_range=(1,2), sublinear_tf=True, dinamik min_df, class_weight='balanced'.
- **Performans iyileştirmesi:** Random Forest için TruncatedSVD ile boyut indirgeme ekleme.
- **Sınıf dengesizliği farkındalığı:** F1-score'un temel metrik olarak seçilmesi, accuracy'nin yanıltıcılığının fark edilmesi.
- **Test stratejisi:** ön işleme ve etiket normalizasyonu için birim test yazılması.
- **Kullanılabilirlik:** otomasyon .bat dosyaları, hata durumunda pencerenin kapanmaması, .venv yokluğunda anlaşılır mesaj.
- **Hata çözümleri:** Bölüm 7'de listelenen altı somut hatanın teşhisi ve giderilmesi.

Prompt'larımı **Rol–Bağlam–Görev–Format** şablonu ve **Zero-shot / Few-shot / Chain-of-Thought** teknikleriyle yapılandırmaya çalıştım; bu sayede AI çıktılarının jenerik kalıplar yerine projemin yapısına yakın biçimde üretilmesini sağladım. Bu çalışma bana, vibecoding'in tek başına yeterli olmadığını; prompt'ların yapılandırılmış biçimde tasarlanmasının da öğrenilmesi gereken ayrı bir pratik olduğunu öğretti.

9. Kaynaklar

1. Almeida, T.A., Hidalgo, J.M.G., Yamakami, A. (2011). *Contributions to the Study of SMS Spam Filtering: New Collection and Results*. ACM Symposium on Document Engineering. UCI Machine Learning Repository — SMS Spam Collection: <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>
2. Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
3. Manning, C.D., Raghavan, P., Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. (TF-IDF ve text classification için temel referans.)
4. Streamlit Documentation — <https://docs.streamlit.io/>
5. Gradio Documentation — <https://www.gradio.app/docs>
6. Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS.

Bu makale, projenin tüm kod ve dokümanlarıyla birlikte GitHub deposunda yer almaktadır. README.md projenin teknik kullanım kılavuzudur; bu makale ise tasarım, gerekçe ve süreç günlüğü olarak okunmalıdır.